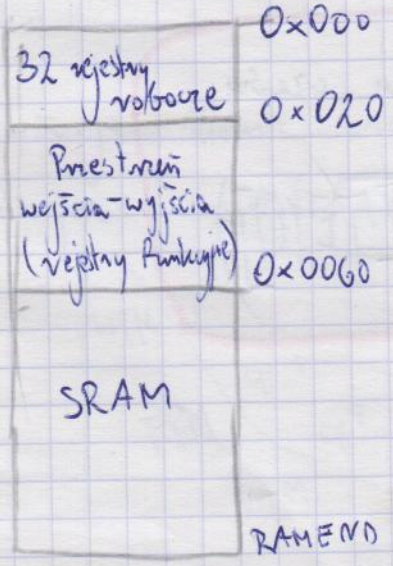
ROZPROSZONE SYSTEMY AKWIZYCJI DANYCH - WYKŁAD

Rodzaje pamięci

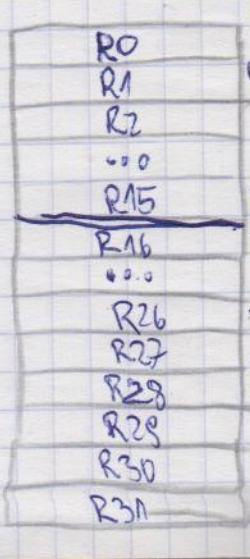
→ układy AVR mają architekturę harwardzką - tzn rozdzielono obszary adresowe pamięci programu i danych

→ obszar pamięci SRAM połączono z rejestrami roboczymi i funkcyjnymi (wspólna przestrzeń adresowa)

→ pamięć EEPROM



Rejestry robocze (ogólnego przeznaczenia)



→ 32-bajtowy obszar

→ jednostka ALU ma bezpośredni dostęp

→ 6 ostatnich rejestrów tworzy trzy pary dwubajtowych rejestrów indeksowych używanych jako wskaźniki w adresowaniu pośrednim

→ tryb pary umożliwiają lepszą optymalizację programów
→ Rejestry R0 - R15 są objęte ograniczeniem - nie mogą być bezpośrednio używane przez ALU
→ Zakazane: ldi, sbci, andi, ori, cpi.

Rejestry funkcyjne (przestrzeń wejścia-wyjścia)

→ dostarczają informacji o aktualnym stanie mikrokontrolera lub umożliwiają sterowanie pracą zdrenia i wewnętrznych układów peryferyjnych

→ w nowszych i bardziej rozbudowanych mikrokontrolerach jest ich więcej

Wewnętrzna pamięć SRAM

→ służy do przechowywania zmiennych programowych (jeśli rejestry robocze nie wystarczają)

→ dostęp: 2 cykle zegarowe

→ NIE MOŻNA przeprowadzać na nich operacji ALU w sposób bezpośredni

→ oprócz funkcji przechowywania danych, pamięć SRAM stanowi także stos mikrokontrolera

→ pamięć stosu służy do tymczasowego przechowywania zmiennych lokalnych oraz do zapamiętywania adresów powrotu (śladów)

→ należy uważać, aby stos nie nadpisał innych zmiennych

Tryby adresowania

TEGO NIE BYŁO NA KOLOKWIUM

Adresowanie pamięci SRAM (danych)

- **bezpośrednie**

```
lds R0, 0x0060   ← Ładuj rejestr R0 zawartością pamięci SRAM spod adresu 0x0060
```

- **pośrednie**

```
ldi XH, 0      }
ldi XL, 0x61   } Ładuj adres 0x61 do rejestrów wskaźnika X
st X, R0       ← zapisz zawartości R0 w SRAM o adresie 0x
```

Adresowanie rejestrów roboczych (ogólnego przeznaczenia)

→ ALU ma dostęp bezpośredni - więc używamy adresowania bezpośr

```
clr R0   ← zeruj rejestr R0
mov R1, R0   ← przenieś rejestr R0 do R1
```

Adresowanie rejestrów funkcyjnych (przestrzeń wejścia-wyjścia)

→ bity zarezerwowane wypełniamy zerami
→ adresowanie bezpośrednie z użyciem in, out
→ używając in, out traktujemy przestrzeń wejścia-wyjścia jako osobną ~~wejścia~~

```
in R0, SREG   ← Ładuj rejestr R0 zawartością rejestru SR
lds R0, SREG+0x20   ← to samo, w normalnym adresowan
```

Adresowanie pamięci programu

→ skoki lub wywołania podprogramów

→ odczyt stałych

→ do odczytu/zapisu z pamięci programu używamy adresowania pośredniego z wykorzystaniem wskaźników Z

→ oprócz tego wykorzystywany jest rejestr RAMPZ (RAM Page Z Select Register) - określa on stronę pamięci programu

→ Adresowanie skoków:

- tryb względny:
  - rjmp - skok ze śladem
  - rcall - skok bez śladu
- tryb bezpośredni:
  - jmp
  - call (potem ret)

Podstawowe rejestry funkcyjne

SREG - Rejestr statusu

→ Stan jego bitów jest uzależniony od wyniku ostatniej operacji wykonanej przez ALU (również operacje porównujące)

→ wykorzystywany m.in przez instrukcje warunkowe

→ należy zaimplementować zapis SREG na stosie przy przechodzeniu do podprogramów

7	6	5	4	3	2	1	0
I	T	H	S	V	N	Z	C

I - zezwolenie na przerwania (globalne) - manipulacja sei/cli

- automatycznie zerowany przy wejściu do programu obsługi przerwania, ustawiany po wyjściu z niego (reti)

T - rejestr bitowy - służy do kopiowania bitów - istotny przy instrukcjach warunkowych

H - znacznik przeniesienia połówkowego

S - znak bitu

V - znacznik przepełnienia U2

N - znacznik wartości ujemnej

Z - znacznik wartości zerowej

C - znacznik przeniesienia

SPH : SPL - Wskaźnik stosu

→ specjalny rejestr, który określa adres w pamięci SRAM pod którym zostanie zapisana kolejna wartość przesyłana na stos

→ wskaźnik stosu ma długość dwóch bajtów, a więc składa się z dwóch rejestrów - SPH (Stack Pointer High) oraz SPL (Stack Pointer Low)

→ zazwyczaj jako podstawę stosu wykorzystuje się komórkę SRAM o najwyższym adresie

Inicjalizacja wskaźnika stosu

```
ldi R16, high(RAMEND)   ← ładuj rejestr R16 MSB adresu końca pamięci SRAM
out SPH, R16            ← inicjalizuj SPH
ldi R16, low(RAMEND)    ← ładuj rejestr roboczy R16 LSB adresu końca SRAM
out SPL, R16            ← inicjalizuj SPL
```

→ Odkładanie danych na stos powoduje dekrementację wskaźnika stosu (push), a ściąganie (pop) - inkrementację

→ W przypadku skoków, na stos jest odkładany licznik programu (PC - Program Counter), który ma długość 2 bajtów



GPIOR - Rejestry Ogólnego Przeznaczenia (General Purpose I/O Repisters)

→ dostępne w najnowszych mikrokontrolerach ATMega
→ zwykle 3 (GPIOR 0...2)
→ możliwe jest ich dowolne wykorzystanie
→ dostęp do nich prowadzony jest na poziomie bitowym (poprzez instrukcje cbi, sbi) lub testowym (sbic, sbis)
→ można w nich przechowywać programowe zmienniki stanu i zmienne globalne

RAMPZ - (RAM Page Z Select Repistr)

→ jest on przedłużeniem rejestru indeksowego Z
→ niezbędny w mikrokontrolerach o dużej pojemności pamięci prog
→ w dokumentacji Atmel są jeszcze: RAMPX, RAMPY
→ za pomocą najmłodszego bitu można wybrać stronę pamięci którą chcemy odczytać rozkazem elpm

System przerwań

→ ułatwia mikrokontrolerowi pracę z wieloma układami peryferyjnymi
→ pozwala na skrócenie czasu przejścia do podprogramów, których wykonanie jest w danej chwili konieczne
→ zwalnia jednostę centralną od ciągłego sprawdzania warunków, od których zależy kolejność wykonywania programów

Aktywność przerwań

→ wszystkie typowe przerwania obsługiwane są pod warunkiem, że zezwolenie globalne jest aktywne (znacznik I = 1 rejestru SREG)
→ instrukcje sei (I=1) i cli (I=0) ułatwiają zarządzanie zezwoleniem
→ po inicjalizacji mikrokontroler domyślnie blokuje przerwania
→ do uaktywnienia konkretnego przerwania konieczne być może odpowiednie ustawienie rejestru sterującego danym modułem

Priorytety i wektory przerwań

→ niebezpieczeństwo: więcej niż jedno żądanie przerwania w danym cyklu zegarowym
→ priorytety przerwań - pozwalają ustalić kolejność
→ wektor przerwań - adres w pamięci programu, pod którym powinien znajdować się podprogram obsługi danego przerwania
→ wektory przerwań znajdują się zwykle w początkowej części pamięci flash i zajmują 1 lub 2 słowa rozkazowe
→ z uwagi na małą pojemność, wektorach przerwań umieszcza się instrukcję skoku do miejsca właściwego podprogramu obsługi przerwania
→ im wyższy wektor przerwań (dalej w pamięci), tym niższy ma priorytet

Przerwanie inicjalizacyjne

→ jest wykonywane podczas ~~włączania~~ inicjalizacji mikrokontrolera
→ ma najwyższy priorytet
→ blokada go nie obejmuje
→ podprogram inicjalizujący można zacząć od adresu 0x00

Przykładowy program

```
.org 0x00
jmp Reset
jmp Ext_int0

Reset:
    ldi R16, high(RAMEND)      } ładowanie
    out SPH, R16               } wskaźnika
    ldi R16, low(RAMEND)       } stosu
    out SPL, R16
    sei                        } odblokowanie przerwań
    in R16, GICR               }
    ori R16, 1<<INT0           } odblokowanie przerwania INT0
    out GICR, R16              }
    jmp Main
```

Znaczniki przerwań

→ każde źródło przerwania ma dedykowany rejestr w przestrzeni adresowej wejścia-wyjścia, który przechowuje tzw. znacznik przerwania
→ informuje on o wystąpieniu żądania obsługi przerwania
→ w momencie obsługi znacznik jest zerowany
→ jeśli przerwania są wyłączone globalnie, ~~tu~~ lub dla niektórych zablokowane, to konieczne jest manualne zerowanie znacznika, aby żadna ~~nie~~ aplikacja go nie wykorzystywała

Obsługa przerwań

1. Wyłączane jest globalne zezwolenie na przerwania ($\overline{I}=0$)
2. Licznik programu zachowywany jest na stosie (4 cykle zegarowe)
3. Wykonywana jest instrukcja umieszczona w wektorze przerwania (zwykle 2/3 cykle)

→ Całość trwa zwykle min. 6-7 cykli zegara.
W przypadku $I=0$, obsługa przerwania jest opóźniana tak długo, dopóki $I=1$.

→ Powrót z podprogramów odbywa się zwykle przy pomocy reti lub ret

reti

1. Zdejmujemy ze stosu adres powrotny ← ret
2. Wykonanie skoku
3. Włączamy globalne zezwolenie na przerwania ($I=1$)

Boot loader

→ Specjalnie wyodrębniony obszar z końca pamięci programu, w którym umieszcza się procedury umożliwiające aktualizację programu głównego

→ BOOTZ - rejestr, którego bity konfiguracyjne umożliwiają określenie wielkości bootloadera

→ BOOTRST - po zaprogramowaniu tego bitu mikrokontroler zacznie wykonywanie programu od sekcji boot loadera

Moduł USART

→ USART - Universal Synchronous and Asynchronous Serial Receiver and Transmitter

→ Układ ułatwiający prowadzenie prostych transmisji szeregow
→ UART - obsługuje tylko tryb asynchroniczny (bez taktowan
→ może być stosowany do obsługi RS232, RS485 itp.
→ najczęściej stosowany w trybie asynchronicznym (bez linii taktującej
→ dwie linie mikrokontrolera TxD (tor danych nadawanych) oraz RxD (tor danych odebranych); opcjonalnie linia taktująca - XC

Ramka transmisyjna

→ może być dowolnie ustawiana

St	0	1	2	3	4	5	6	7	8	P

- bit startu - rozpoczyna transmisję
- bity przesyłanego słowa (5-8 bitów)
- bit parzystości (opcjonalnie)
- bit/y stopu (możliwe dwa)

Dedykowane rejestry funkcyjne

- bufor danych UDR - jego zapisanie rozpoczyna transmisję ramk a odczyt powoduje uzyskanie ostatniej ramki odebranej.
- (UWAGA! Do zapisu wymagany jest znacznik UDRE=1 w UCSRA
- TXB8, RXB8 - przedłużenia rejestrów UDR, jeśli wybrano 9-bitową długość słowa

- UCSRA, UCSRB, UCSRC – ustawienia, kontrola stanu i sterowanie
  - RXC – znacznik odebrania danych
  - TXC – znacznik przesłania danych
  - UDRE – gotowość rejestru UDR (empty)
  - FE – błąd ramki transmisyjnej
  - DOR – przepełnienie bufora wejściowego
  - PE – błąd parzystości
  - U2X – podwójna prędkość trans.
  - MPCM
  - RXCIE
  - TXCIE
  - UDRIE
  - RXEN, ... i inne, ...
- UBRRH, UBRRL – rejestry służące do określania prędkości transmisji

Przykład – Oczekiwanie na gotowość i nadanie słowa

```
Czekaj:
    sbis UCSRA, UDRE    ← jeśli UDRE==1, to pomiń nast. rozkaz
    rjmp Czekaj

    ldi R16, 0xAB       ← ładuj 0xAB do rejestru R16
    out UDR, R16        ← rozpocznij transmisję, wyślij R16 do UBR
```

Aby nadać słowo przez USART, wystarczy umieścić je w rejestrze UDR. Wymagany UDRE=1 w UCSRA.

Odbiór danych

1. Sprawdzamy znacznik RXC, który informuje o zakończeniu transmisji
2. Włączamy obsługę przerwań związanych z odebraniem danych (RXCIE)
3. Sprawdzamy czy nie ma błędów (znaczniki PE, FE, DOR)
4. Odczytujemy odebrane słowo z rejestru UDR.

```
Czekaj:
    sbis UCSRA, RXC     ← jeśli RXC=1, pomiń następny rozkaz
    rjmp Czekaj
    in R16, UCSRA       ← ładuj UCSRA do R16
    in R17, UDR         ← ładuj UDR do R17
    andi R16, (1<<FE)|(1<<DOR)|(1<<PE)  ← jeśli żaden ze znaczników błędu nie jest ustawiony, to skocz do Dane_poprawne
    breq Dane_poprawne
```

ldi R16, 0xFF ← ustaw informacja o błędzie

Dane poprawne
...

Tryb synchroniczny

→ w tym trybie wykorzystywana jest linia taktująca XCK
⇒ wybór trybu synchronizacji - znacznik UMSEL w ~~USRC~~ UCS
→ sygnał zegarowy może działać niezależnie lub być podporządkowany układom zewnętrznym
→ bit DDxn - wybór między trybem nadrzędnym a podrzędnym
⇒ w trybie nadrzędnym ustawiamy prędkość transmisji w UBRRH i UBRRL
⇒ częstotliwość taktowania powinna być mniejsza niż czwarta część zegara systemowego

Moduł SPI

→ SPI - (Serial Peripheral Interface) - szybki, dupleksowy, synchroniczny interfejs szeregowy

→ pozwala na łączenie jednego układu nadrzędnego z wieloma urządzeniami podrzędnymi

4 wyprowadzenia:

- MOSI (Master Out, Slave In) } linie danych układu nadrzędnego i podrzędnego
- MISO (Master In, Slave Out) - }
- SCK (Serial Clock) - wejście (tryb nadrzędny) lub wejście (tr. podrzędny) syg. taktu
- $\overline{SS}$ (Slave Select) - aktywuje wybrane urządzenie podrzędne